

ESTUDIANTE : JHON GILMER MAMANI APAZA

TRABAJO ENCARGADO N6-UNIDAD_2

Trabajo 1

los docentes de la facultad de estadística e informática y con su respectivo “índice h”

NUMERO	NOMBRES	APELLIDOS	índice h	documentos	cita de documentos
001	Percy	HUATA PANCA	3	8	11
002	Fred	Torres-Cruz	4	40	0
003	Elqui yeye	PARI CONDORI	2	4	4
004	José pánfil	TITO LIPA	2	5	7
005	Remo	CHOQUEJAHUA ACERO	4	4	10
006	Vladimiro	IBAÑEZ QUISPE	5	21	52
007	Juan carlos	JUAREZ VARGAS	1	3	8
008	Milton ant	LOPEZ CUEVA	1	6	4
009	Ernesto na	TUMI FIGUEROA	3	8	23
010	Romel P.	Melgarejo-Bolívar	3	3	6
011	Fredy heric	VILLASANTE SARAVIDA	3	9	21
012	Juan reyna	PAREDES QUISPE	1	0	0
013	Leonel	COYLA DME	1	5	8
014	Charles igr	MENDOZA MOLLOCON	3	8	17
015	Ángel	Javier Quispe Carita	1	8	9
016	Alejandro	APAZA TARQUI	1	3	4
017	Bernabé	CANQUI FLORES	2	9	20
018	Edgar eloy	CARPIO VARGAS	3	8	27
019	Leonid	ALEMÁN GONZALES	2	4	14

Los docentes que puse en la lista los busqué en **Scopus**

(<https://www.scopus.com/home.uri?zone=header&origin=AuthorNamesList>), donde se puede ver el **índice h** y la **cantidad de documentos que han publicado** durante su trayectoria. Además, como parte de la práctica, tengo que poner una **captura de pantalla que muestre que me inscribí en la página de CONCYTEC** (<https://ctivitaec.concytec.gob.pe/appDirectorioCTI/index.jsp>).

The screenshot shows the 'PERFIL' (Profile) section of the CONCYTEC registration system. At the top, there are navigation tabs: 'INICIO', 'SOLICITA CALIFICACIÓN', 'RENACOT', and a user profile dropdown for 'JHON GILMER MAMANI APAZA'. The main content area includes a 'Cargando foto' (Loading photo) status, a 'Calificación, Clasificación y Registro de Investigadores' section with a 'Solicitar Incorporación' button, and a 'Selección de archivos' section with a 'Agregar foto' button. Below these, there is a 'DATOS PERSONALES' section with fields for 'Nombres' (Jhon Gilmer) and 'Apellido paterno' (Mamani).

Trabajo 2

El segundo trabajo consiste en identificar **al menos dos docentes** que tengan un **índice h superior a 10**, realizando la búsqueda tanto a **nivel nacional como internacional**. Además, se debe presentar **uno de los documentos** que cada docente haya publicado como ejemplo de su producción académica.

(<https://ctivitae.concytec.gob.pe/appDirectorioCTI/index.jsp>).

Primero

Miguel Ángel [Chávez Fumagalli](#) Univ. Católica de Santa María (Arequipa) Ingeniería y Tecnología / Informática.

Chávez-Fumagalli, Miguel Ángel

[Universidad Católica de Santa María](#), Arequipa, Perú • Identificación de Scopus: 35271437900 •  [0000-0002-8394-4802](#) ↗

[Mostrar toda la información](#)

2.656	155	29
Citas de 1.297 documentos	Documentos	Índice h

Segundo

Dr. Walter [Curioso Vélchez](#) Universidad Continental Informática Médica, Salud Pública, Sistemas de Información. Investigador de alto impacto, figura en el ranking del Top 2% de científicos más citados del mundo (con un h-index muy alto).

Curioso, Walter H.

[Universidad Continental](#), Huancayo, Perú • Identificación de Scopus: 6602404688 •  [0000-0003-3789-7483](#) ↗

[Mostrar toda la información](#)

1.868	93	22
Citas de 1.570 documentos	Documentos	Índice h

 [Editar perfil](#)  [Más](#)

Sus publicaciones

	Autor	Documentos	Afiliación	Ciudad	País/Territorio
☐ 1	Curioso, Walter H.	93	Universidad Continental	Huancayo	Perú
	Ver último título ▼				
☐ 2	Curioso-Vélchez, Iván Carlos	3	Universidad Científica del Sur	Lima	Perú
	Ver último título ▼				

Mostrar:

20 

resultados por página

1 

[Parte superior de la página](#)

Trabajo 3

4 metodos

1. METODO FALSA POSICION
2. METODO DE LA SECANTE
3. METODO DEL PUNTO FIJO
4. METODO DE LA BISECCION

1. - METODO FALSA POSICION

El **método de la falsa posición** es un procedimiento numérico que aproxima la raíz de una función trazando una recta entre dos puntos donde la función cambia de signo y tomando la intersección con el eje x como nueva estimación.

Codigo

```
import math

def falsa_posicion(f, a, b, tol=1e-6, max_iter=100):
    print("\n----- MÉTODO DE LA FALSA POSICIÓN -----")
    print(f"{'Iter':<6} {'a':<12} {'b':<12} {'c':<12} {'f(c)':<15} {'Error':<12}")
    print("-" * 70)

    if f(a) * f(b) > 0:
        print("Intervalo inválido: f(a) y f(b) deben tener signos opuestos.")
        return None

    c_ant = a
    for i in range(1, max_iter + 1):
        fa, fb = f(a), f(b)
        c = (a * fb - b * fa) / (fb - fa)
        fc = f(c)
        error = abs(c - c_ant) if i > 1 else abs(b - a)
        print(f"{'i':<6} {'a':<12.6f} {'b':<12.6f} {'c':<12.6f} {'fc':<15.6e} {'error':<12.6e}")

        if abs(fc) < tol or error < tol:
            print(f"\n✓ Convergíó en {i} iteraciones → Raíz ≈ {c:.8f}")
            return c
        if fa * fc < 0:
            b = c
        else:
            a = c
        c_ant = c
    print("\n✗ No convergió.")
    return c

f = lambda x: math.exp(-x) - x
falsa_posicion(f, 0, 1)
```

Salida esperada

Iter	a	b	c	f(c)	Error
1	0.000000	1.000000	0.612700	-7.081395e-02	1.000000e+00
2	0.000000	0.612700	0.572181	-7.888273e-03	4.051842e-02
3	0.000000	0.572181	0.567703	-8.773920e-04	4.478198e-03
4	0.000000	0.567703	0.567206	-9.757273e-05	4.976616e-04
5	0.000000	0.567206	0.567150	-1.085062e-05	5.533839e-05
6	0.000000	0.567150	0.567144	-1.206646e-06	6.153865e-06
7	0.000000	0.567144	0.567143	-1.341853e-07	6.843412e-07

✓ Convergíó en 7 iteraciones → Raíz ≈ 0.56714338

2. -METODO DE LA SECANTE

El **metodo de la secante** es un procedimiento iterativo para hallar raíces de una función $f(x)$ usando dos aproximaciones iniciales. En cada paso, se traza la recta secante entre los puntos $(x_{n-1}, f(x_{n-1}))$ y $(x_n, f(x_n))$; el punto donde esta recta corta el eje xxx se toma como la nueva aproximación.

No requiere derivadas y suele converger más rápido que el método de bisección, aunque puede fallar si los valores iniciales no están bien elegidos o si el denominador se acerca a cero. Su convergencia es **superlineal**, con un orden cercano a 1.618.

Codigo

```
def secante(f, x0, x1, tol=1e-6, max_iter=100):
    print("\n----- MÉTODO DE LA SECANTE -----")
    print(f"{'Iter':<6} {'x_{n-1}':<12} {'x_n':<12} {'x_{n+1}':<12} {'f(x_{n+1})':<15} {'Error':<12}")
    print("_" * 70)

    for i in range(1, max_iter + 1):
        fx0, fx1 = f(x0), f(x1)
        if abs(fx1 - fx0) < 1e-12:
            print("Error: división por cero.")
            return None
        x2 = x1 - fx1 * (x1 - x0) / (fx1 - fx0)
        fx2 = f(x2)
        error = abs(x2 - x1)
        print(f"{'i':<6} {'x0':<12.6f} {'x1':<12.6f} {'x2':<12.6f} {'fx2':<15.6e} {'error':<12.6e}")
        if abs(fx2) < tol or error < tol:
            print(f"\n✓ Convergíó en {i} iteraciones → Raíz ≈ {x2:.8f}")
            return x2
        x0, x1 = x1, x2
    print("\nX No convergió.")
    return x2

f = lambda x: x**3 - 5*x + 3
secante(f, 0, 1)
```

Salida

Iter	x _{n-1}	x _n	x _{n+1}	f(x _{n+1})	Error
1	0.000000	1.000000	0.750000	-3.281250e-01	2.500000e-01
2	1.000000	0.750000	0.627907	1.080282e-01	1.220930e-01
3	0.750000	0.627907	0.658147	-5.655480e-03	3.024050e-02
4	0.627907	0.658147	0.656643	-8.398340e-05	1.504389e-03
5	0.658147	0.656643	0.656620	6.826795e-08	2.267680e-05

✓ Convergíó en 5 iteraciones → Raíz ≈ 0.65662041

En las iteraciones del método de la secante, los valores se fueron acercando progresivamente a la raíz real de la función. En cada paso, la nueva aproximación fue más precisa que la anterior, mostrando una convergencia rápida. Finalmente, tras pocas iteraciones, el método alcanzó un valor estable de xxx donde $f(x)f(x)f(x)$ se aproximó a cero, indicando que se encontró la raíz con el error deseado.

3. - METODO DEL PUNTO FIJO

El **método de Newton-Raphson** es un procedimiento iterativo que usa la función y su derivada para aproximar una raíz. A partir de un valor inicial x_0 , se aplica la fórmula

hasta que el resultado converge. Es un método rápido y preciso, pero puede fallar si la derivada es muy pequeña o si el punto inicial no está cerca de la raíz.

Codigo

```
def punto_fijo(g, x0, tol=1e-6, max_iter=100):
    print("\n----- MÉTODO DEL PUNTO FIJO -----")
    print(f"{'Iter':<6} {'x_n':<15} {'x_{n+1}':<15} {'Error':<15}")
    print("-" * 55)

    for i in range(1, max_iter + 1):
        x1 = g(x0)
        error = abs(x1 - x0)
        print(f"{'i':<6} {'x0':<15.8f} {'x1':<15.8f} {'error':<15.8e}")
        if error < tol:
            print(f"\n✓ Convergíó en {i} iteraciones → Raíz ≈ {x1:.8f}")
            return x1
        x0 = x1
    print("\n✗ No convergíó.")
    return x1

import math
g = lambda x: math.cos(x)
punto_fijo(g, 0.5)
```

Salida

Iter	x_n	x_{n+1}	Error
1	0.500000000	0.87758256	3.77582562e-01
2	0.87758256	0.63901249	2.38570068e-01
3	0.63901249	0.80268510	1.63672607e-01
4	0.80268510	0.69477803	1.07907074e-01
5	0.69477803	0.76819583	7.34178045e-02
6	0.76819583	0.71916545	4.90303853e-02
7	0.71916545	0.75235576	3.31903135e-02
8	0.75235576	0.73008106	2.22746963e-02
9	0.73008106	0.74512034	1.50392782e-02
10	0.74512034	0.73500631	1.01140323e-02
11	0.73500631	0.74182652	6.82021363e-03
12	0.74182652	0.73723573	4.59079720e-03
13	0.73723573	0.74032965	3.09392644e-03
14	0.74032965	0.73824624	2.08341355e-03
15	0.73824624	0.73964996	1.40372444e-03
16	0.73964996	0.73870454	9.45423413e-04
17	0.73870454	0.73934145	6.36912924e-04
18	0.73934145	0.73891245	4.29002949e-04
19	0.73891245	0.73920144	2.88994804e-04
20	0.73920144	0.73900678	1.94664355e-04
21	0.73900678	0.73913791	1.31130981e-04
22	0.73913791	0.73904958	8.83301671e-05
23	0.73904958	0.73910908	5.95008253e-05
24	0.73910908	0.73906900	4.00802165e-05
25	0.73906900	0.73909600	2.69986317e-05
26	0.73909600	0.73907781	1.81865506e-05
27	0.73907781	0.73909006	1.22507031e-05
28	0.73909006	0.73908181	8.25221014e-06
29	0.73908181	0.73908737	5.55879293e-06
30	0.73908737	0.73908363	3.74446756e-06
31	0.73908363	0.73908615	2.52231940e-06
32	0.73908615	0.73908445	1.69906423e-06
33	0.73908445	0.73908559	1.14451031e-06
34	0.73908559	0.73908482	7.70955819e-07

✓ Convergíó en 34 iteraciones → Raíz ≈ 0.73908482

4. - METODO DE LA BISECCION

El método de bisección es un procedimiento numérico que encuentra una raíz de $f(x)=0$ en un intervalo $[a,b]$ donde ocurre un cambio de signo. En cada paso divide el intervalo a la mitad y selecciona el subintervalo donde la función sigue cambiando de signo. Es lento pero muy seguro, ya que siempre converge si la función es continua y cumple $f(a) \cdot f(b) < 0$.

CODIGO

```
def biseccion(f, a, b, tol=1e-6, max_iter=100):
    print("\n----- MÉTODO DE LA BISECCIÓN -----")
    print(f"{'Iter':<6} {'a':<12} {'b':<12} {'c':<12} {'f(c)':<15} {'Error':<12}")
    print("_" * 70)

    if f(a) * f(b) > 0:
        print("Intervalo inválido: f(a) y f(b) deben tener signos opuestos.")
        return None

    for i in range(1, max_iter + 1):
        c = (a + b) / 2
        fc = f(c)
        error = abs(b - a)
        print(f"{'i':<6} {'a':<12.6f} {'b':<12.6f} {'c':<12.6f} {'fc':<15.6e} {'error':<12.6e}")

        if abs(fc) < tol or error < tol:
            print(f"\n✅ Convergió en {i} iteraciones → Raíz ≈ {c:.8f}")
            return c
        if f(a) * fc < 0:
            b = c
        else:
            a = c
    print("\n❌ No convergió.")
    return c

f = lambda x: x**3 + 4*x**2 - 10
biseccion(f, 1, 2)
```

SALIDA

----- MÉTODO DE LA BISECCIÓN -----					
Iter	a	b	c	f(c)	Error
1	1.000000	2.000000	1.500000	2.375000e+00	1.000000e+00
2	1.000000	1.500000	1.250000	-1.796875e+00	5.000000e-01
3	1.250000	1.500000	1.375000	1.621094e-01	2.500000e-01
4	1.250000	1.375000	1.312500	-8.483887e-01	1.250000e-01
5	1.312500	1.375000	1.343750	-3.509827e-01	6.250000e-02
6	1.343750	1.375000	1.359375	-9.640884e-02	3.125000e-02
7	1.359375	1.375000	1.367188	3.235579e-02	1.562500e-02
8	1.359375	1.367188	1.363281	-3.214997e-02	7.812500e-03
9	1.363281	1.367188	1.365234	7.202476e-05	3.906250e-03
10	1.363281	1.365234	1.364258	-1.604669e-02	1.953125e-03
11	1.364258	1.365234	1.364746	-7.989263e-03	9.765625e-04
12	1.364746	1.365234	1.364990	-3.959102e-03	4.882812e-04
13	1.364990	1.365234	1.365112	-1.943659e-03	2.441406e-04
14	1.365112	1.365234	1.365173	-9.358473e-04	1.220703e-04
15	1.365173	1.365234	1.365204	-4.319188e-04	6.103516e-05
16	1.365204	1.365234	1.365219	-1.799489e-04	3.051758e-05
17	1.365219	1.365234	1.365227	-5.396254e-05	1.525879e-05
18	1.365227	1.365234	1.365231	9.030993e-06	7.629395e-06
19	1.365227	1.365231	1.365229	-2.246580e-05	3.814697e-06
20	1.365229	1.365231	1.365230	-6.717413e-06	1.907349e-06
21	1.365230	1.365231	1.365230	1.156788e-06	9.536743e-07

✅ Convergió en 21 iteraciones → Raíz ≈ 1.36523008